MACHINE LEARNING // SESSION 5

Machine Learning

Model Training

& Classification

KNN · Decision Trees · Random Forest · XGBoost

What We'll Cover Today

A complete deep-dive into Machine Learning classification algorithms

01 **The ML Pipeline**

Where model training fits in the workflow

02 **Train / Test Split**

Why we split data and how to do it correctly

03 **Evaluation Metrics**

Accuracy, Precision, Recall, F1, Confusion Matrix

04 **K-Nearest Neighbors**

Concept, math, choosing K, pros & cons

05 **Decision Trees**

How trees split, Gini impurity, visualization

06 **Random Forest**

Ensemble learning, bagging, feature importance

07 **XGBoost**

Boosting, learning rate, why it wins

08 **Model Comparison**

How to choose the best model

The Machine Learning Pipeline

Where does model training fit? Let's zoom out first.

Data Collection — EDA — Preprocessing — Train/Test Split — **Model Training** — Evaluation — Deploy

Already Done

Data Collection, EDA, Cleaning, Encoding, Scaling

TODAY'S FOCUS

Train/Test Split, Model Training, Evaluation & Comparison

Coming Next

Hyperparameter Tuning, Cross-Validation, Deployment

Train / Test Split

The golden rule: never evaluate a model on data it has already seen

TRAINING SET — 75%

≈ 668 passengers used to teach the model

TEST — 25%

≈ 223 unseen passengers

🏫 Real-Life Analogy

Think of it like preparing for an exam:

- Training set = the practice problems you study
- Test set = the REAL exam questions
- If you memorize answers instead of learning, you'll fail the real exam → this is called OVERFITTING

⚠️ Key Rules

✅ random_state=42 → same split every run
✅ stratify=y → same class % in both sets
✅ Fit scaler on train only → no data leakage
❌ Never test on training data
❌ Never 'look at' test data during training

```
X_train, X_test, y_train, y_test = train_test_split(X, y,  test_size=0.25,  random_state=42,  stratify=y)
```

Evaluation Metrics — The Confusion Matrix

Before we build models, we need to know how to MEASURE them

	Predicted: NO	Predicted: YES
Actual: NO	**TN** True Negative 118	**FP** False Positive 16
Actual: YES	**FN** False Negative 25	**TP** True Positive 64

Accuracy — 82.5%

`(TP + TN) / Total`

Overall correct predictions

Precision — 80%

`TP / (TP + FP)`

Of predicted survivors: how many did survive?

Recall — 72%

`TP / (TP + FN)`

Of actual survivors: how many did we catch?

F1-Score — 76%

`2 × (P × R) / (P + R)`

Harmonic mean of Precision and Recall

Precision vs Recall — The Trade-off

Understanding when each metric matters most

Precision

`TP / (TP + FP)`

"Of all the people I predicted survived — how many actually did?"

High Precision = Few False Alarms

Example: Spam filter

You don't want to mark real emails as spam (avoid FP).
→ Prioritize PRECISION

Recall (Sensitivity)

`TP / (TP + FN)`

"Of all the people who actually survived — how many did I catch?"

High Recall = Few Missed Cases

Example: Cancer screening

You don't want to MISS a sick patient (avoid FN).
→ Prioritize RECALL

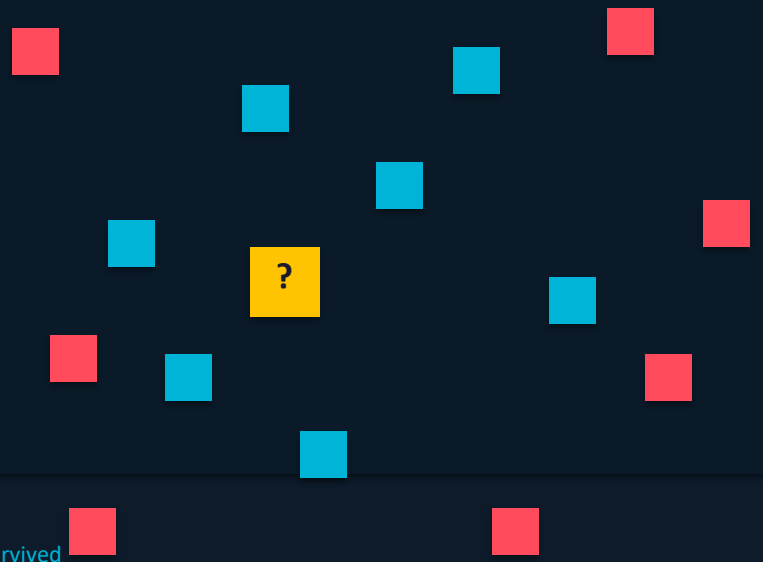
F1-Score combines both: F1 = 2 × (Precision × Recall) / (Precision + Recall) — Use F1 when you care equally about both

K-Nearest Neighbors (KNN)

"Tell me who your neighbors are, and I'll tell you who you are"

K=3 nearest:

2 survived (🟦) + 1 not (🟥)
→ PREDICT: Survived



Survived

Not Survived

New passenger (to predict)

Step 1

Calculate the distance from the new point to ALL training points

Step 2

Sort by distance → pick the K closest (K=5 by default)

Step 3

Count votes: majority class among K neighbors wins

Step 4

Output the predicted class

KNN — Choosing K & Key Considerations

K = 1 — **Extreme Overfitting**

Memorizes every training point. Noisy, unstable.

K = 5 (typical) — **Good Balance**

Generalizes well. Stable predictions.

K = 100 — **Extreme Underfitting**

Ignores local patterns. Too simple.

→ Smaller K = more complex boundary = overfitting | Larger K = smoother = underfitting

✓ Advantages

- Simple and intuitive to understand
- No training phase — just stores data
- Works well with small datasets
- Naturally handles multi-class problems

✗ Disadvantages

- Slow at prediction (calculates ALL distances)
- Needs feature scaling (sensitive to magnitude)
- Fails with high-dimensional data (curse)
- Memory-intensive — stores entire dataset

Decision Trees

A flowchart of yes/no questions that leads to a prediction

Is Sex = female?

YES

Is Pclass = 1 or 2?

YES

✓ **Survived**

NO

✗ **Did Not Survive**

NO

Is Age < 10?

YES

✓ **Survived**

NO

✗ **Did Not Survive**

How does the tree decide WHERE to split?

The tree tries every possible split and picks the one that creates the PUREST groups (best separates survivors from non-survivors).

It uses GINI IMPURITY to measure purity: $Gini = 1 - \Sigma(p_i^2)$ → Gini = 0 means perfectly pure (all one class) | Gini = 0.5 means totally mixed

Gini Impurity — How Trees Make Splits

The algorithm finds the split that minimizes Gini impurity in child nodes

$$Gini(node) = 1 - [\ p(survived)^2 + p(not\ survived)^2\ ]$$

Pure Node

50 survived,
0 not survived

```
Gini = 1 – [(1.0)² + (0.0)²]
Gini = 0.0
```

✅ **Perfect split!**

100% survived → no mixing

Mixed Node

25 survived,
25 not survived

```
Gini = 1 – [(0.5)² + (0.5)²]
Gini = 0.5
```

❌ **Worst possible**

50/50 → maximum impurity

Partial Node

40 survived,
10 not survived

```
Gini = 1 – [(0.8)² + (0.2)²]
Gini = 0.32
```

🔶 **Acceptable**

Mostly one class

Key Hyperparameters: max_depth — limits tree depth (prevents overfitting) | min_samples_leaf — min samples per leaf | random_state — reproducibility

Overfitting vs Underfitting

The most important concept in machine learning — every model must balance these two

Underfitting (High Bias)

Model is TOO SIMPLE.
Doesn't learn enough from data.

- Low accuracy on BOTH train and test
- Model ignores important patterns
- Happens with small trees or large K
- Fix: increase model complexity

```
Training: 60%   Test: 58%
```

Good Fit (Just Right)

Model captures real patterns.
Generalizes to new data.

- High accuracy on training data
- High (similar) accuracy on test
- The GOAL we're always aiming for
- Fix: hyperparameter tuning

```
Training: 85%   Test: 83%
```

Overfitting (High Variance)

Model is TOO COMPLEX.
Memorizes training data.

- Very high train accuracy
- Much lower test accuracy
- Happens with deep/full trees
- Fix: regularization, max_depth

```
Training: 99%   Test: 72%
```

← More Complex Model →

Random Forest

"The wisdom of the crowd" — combining many imperfect trees into one powerful model

Training Data (891 passengers)

↓ bootstrap sample

↓ bootstrap sample

↓ bootstrap sample

Tree 1

Random subset of features

→ **Survived**

Tree 2

Random subset of features

→ **Did Not**

Tree 3

Random subset of features

→ **Survived**

↓ Majority Vote (2 Survived : 1 Not)

Final Prediction: SURVIVED

Bootstrap Sampling (Bagging)

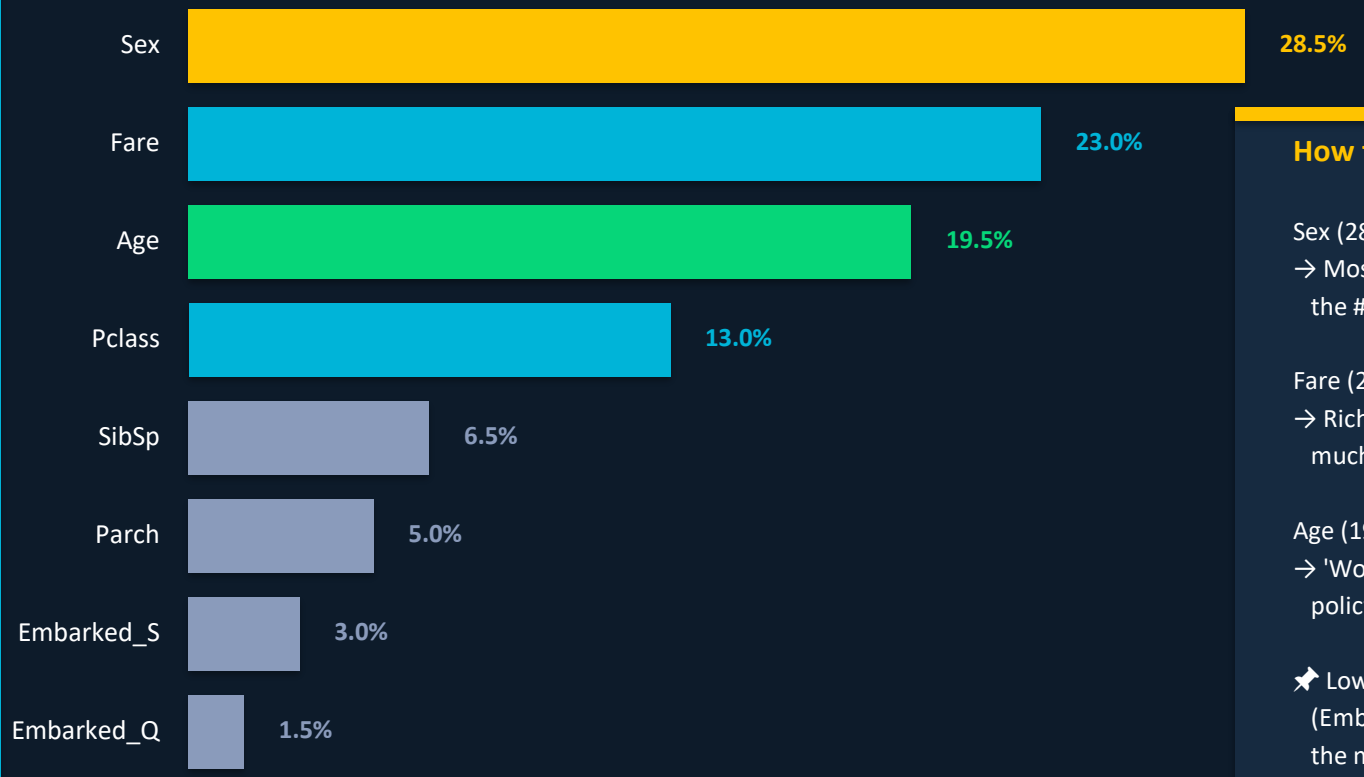
Each tree trains on a random sample WITH replacement from the training data. Some rows are selected twice, others not at all. This creates DIVERSE trees.

Random Feature Selection

At each split, only a RANDOM SUBSET of features is considered (typically √n features). This forces diversity between trees and prevents any one feature from dominating.

Random Forest — Feature Importance

A powerful bonus: Random Forest tells us WHICH features matter most



How to Read This

Sex (28.5%):
→ Most important! Gender was the #1 survival predictor

Fare (23%):
→ Richer passengers had much higher survival rates

Age (19.5%):
→ 'Women and children first' policy shows in the data

📌 Low importance features (Embarked) barely helped the model — we could drop them to simplify.

XGBoost — Extreme Gradient Boosting

Build trees SEQUENTIALLY — each tree fixes the errors of the previous one

Tree 1

Predicts based on raw data

Errors = 89 passengers wrong

→

Tree 2

Focuses on what Tree 1 got wrong

Errors = 47 passengers wrong

→

Tree 3

Focuses on what Trees 1+2 missed

Errors = 28 passengers wrong

→

Tree N (100 total)

Keeps correcting residual errors

Errors = 18 passengers wrong

Final Prediction = Weighted Sum of ALL 100 Trees → Much more accurate than any single tree!

`n_estimators=100`

Number of trees to build
More = better but slower

`max_depth=3`

How deep each tree is
Shallow trees = less overfit

`learning_rate=0.1`

How much each tree corrects
Lower = more careful, often better

`random_state=42`

Reproducibility seed
Same result every run

Random Forest vs XGBoost

Both are ensemble methods — but they work very differently

Aspect	🌳 Random Forest	⚡ XGBoost
Method	Bagging (parallel trees)	Boosting (sequential trees)
Tree type	Deep trees (low bias)	Shallow trees (stumps)
Speed	Fast training (parallel)	Slower (sequential)
Overfitting	Resistant by design	Can overfit → tune carefully
Best for	Noisy data, fast experiments	High accuracy competitions
Titanic acc.	~81.6%	~82.5%

💡 Practical tip: Start with Random Forest (faster, simpler), then try XGBoost to squeeze out extra accuracy

Feature Scaling

Why not all features start equal — and how we fix it

✗ The Problem (Before Scaling)

```
Age range:  0  →  80
Fare range: 0  →  512

When KNN calculates distance:
  dist = √( Δage² + Δfare² )

Fare is 6× larger → it DOMINATES
the distance calculation completely.
Age becomes almost invisible!
```

✓ The Solution (After Scaling)

```
StandardScaler transforms to:
  Mean = 0,  Std Dev = 1

Age:  22 → (22-29.7)/14.5  = -0.53
Fare:  7 → (7-32.2)/49.7   = -0.51

Now both contribute EQUALLY
to the distance calculation.
Fair competition!
```

StandardScaler (Z-score normalization)

Formula: $z = (x - \text{mean}) / \text{std_dev}$
Output: mean=0, std=1
Use when: data roughly follows normal distribution
Best for: KNN, SVM, Logistic Regression

MinMaxScaler (Min-Max normalization)

Formula: $x' = (x - \min) / (\max - \min)$
Output: range [0, 1]
Use when: data has a known fixed range
Best for: Neural Networks, image data

⚠ Data Leakage — A Critical Warning

The most common mistake beginners make — it silently inflates your results

✗ WRONG — Data Leakage!

```
scaler = StandardScaler()

# FIT on ALL data (WRONG!)
X_scaled = scaler.fit_transform(X)

# Now split
X_train, X_test = train_test_split(X_scaled)
```

Why is this wrong?
The scaler has already SEEN the test data
when computing the mean and std.
It's like giving answers before the exam!
Your reported accuracy is unrealistically high.

✓ CORRECT — No Leakage!

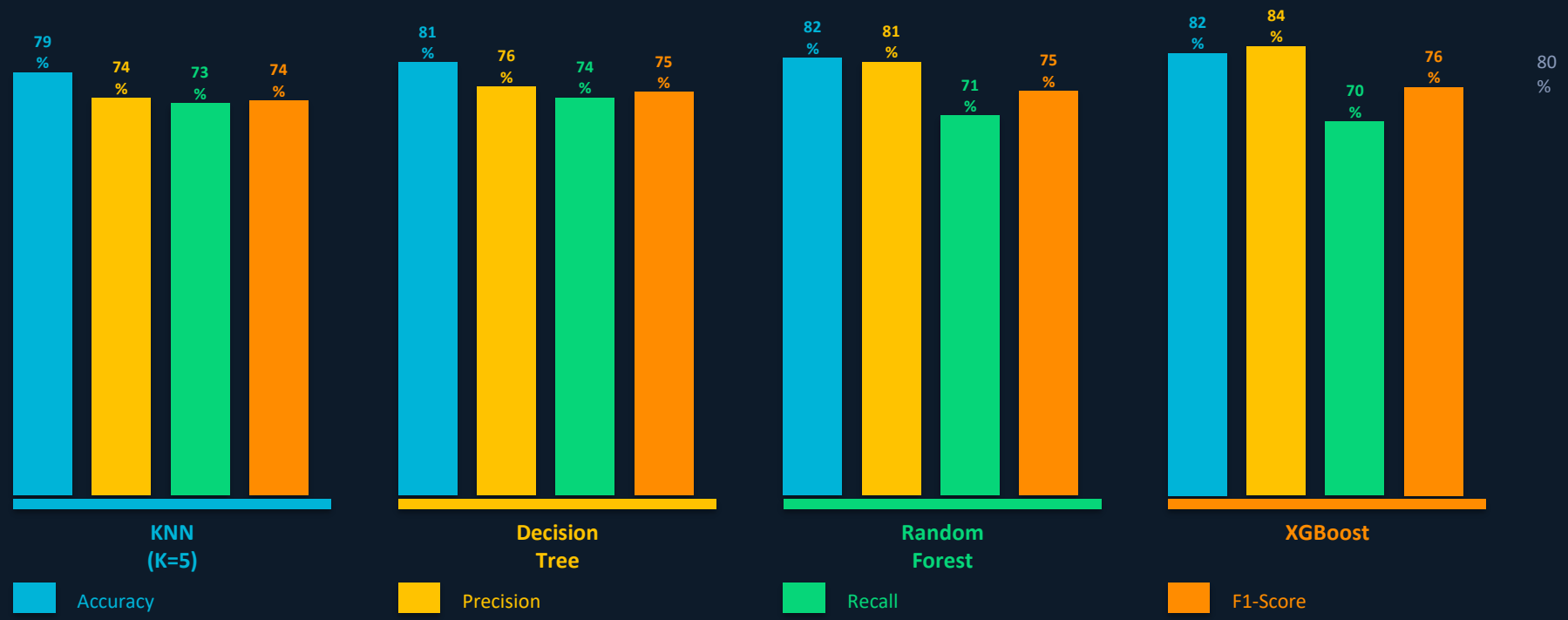
```
# Split FIRST
X_train, X_test = train_test_split(X)

scaler = StandardScaler()
# Fit on train ONLY
X_train = scaler.fit_transform(X_train)
# Apply to test
X_test  = scaler.transform(X_test)
```

Why is this correct?
The scaler only learns from training data.
The test data is truly 'unseen' during training.
Your accuracy reflects REAL performance.

🔑 THE GOLDEN RULE: NEVER fit any transformer (scaler, encoder, imputer) on test data. Always fit on train, then ONLY transform the test.

Model Comparison — Titanic Results

How did our 4 models perform on the same test set?



How to Choose the Right Model

There is no single 'best' algorithm — it depends on your situation

Data Size	If: Small (<1K rows) → **KNN or Decision Tree**	If: Large (>100K) → **XGBoost or Random Forest**
Interpretability	If: Need to EXPLAIN the model → **Decision Tree (visual)**	If: Just need accuracy → **XGBoost**
Speed	If: Need fast training → **Random Forest (parallel)**	If: Can wait for accuracy → **XGBoost**
Feature Scaling	If: No time to scale → **Tree-based models**	If: Willing to scale → **KNN (requires scaling)**
Priority	If: High Recall critical → **Tune any model for recall**	If: High Precision needed → **Tune threshold or cost**

Session Summary — Quick Reference

Everything you need to remember from today's session

Train/Test Split

split(test=0.25, stratify=y) | NEVER test on training data

Confusion Matrix

TN / FP / FN / TP | Accuracy, Precision, Recall, F1

KNN

Distance to K neighbors → majority vote | Needs scaling

Decision Tree

Yes/No questions on features | Uses Gini impurity to split

Overfit/Underfit

max_depth, min_samples_leaf → control complexity

Random Forest

50-100 trees + bootstrap + random features → majority vote

XGBoost

Sequential trees, each fixes last errors | learning_rate

Feature Scaling

fit_transform(X_train) + transform(X_test) | NO leakage

Model Selection

XGBoost best accuracy | Tree models best interpretability